GESTION DE PARC AUTOMOBILE

Document	*Cahier des charges fonctionnel et technique*
Version	*1.0 — 09/08/2026*
Périmètre	*Plateforme web de gestion de parc automobile*
Statut	*Validé*

Sommaire

1. Contexte et présentation du projet
2. Objectifs et enjeux
3. Périmètre fonctionnel — les modules
4. Exigences fonctionnelles détaillées
5. Exigences non fonctionnelles
6. Architecture technique et technologie
7. Modèle de données
8. API REST
9. Design, ergonomie et expérience utilisateur
10. Déploiement et exploitation
11. Assurance qualité et tests
12. Planning prévisionnel
13. Évolutions futures
14. Risques et mitigations
15. Structure du projet

1. Contexte et présentation du projet

La gestion d'une flotte de véhicules (camions, utilitaires, berlines) exige le suivi simultané de multiples données : immatriculations, échéances administratives (assurance, vignette, contrôle technique), kilométrage, maintenances, contrôles, conducteurs et documents. Sans outil centralisé, ces informations se dispersent dans des fichiers, des mails ou des cahiers, ce qui génère des oublis coûteux (échéances passées, véhicules immobilisés) et une perte de temps importante.

NexFleet est une plateforme web complète de gestion de parc automobile qui centralise l'ensemble de ces données dans une interface unique, moderne et sécurisée. Elle s'adresse aux flottes de petite et moyenne taille (5 à 200 véhicules) ainsi qu'aux départements de gestion de flotte souhaitant numériser leurs opérations.

Produit	*NexFleet — Gestion de parc automobile*
Type	*Application web (SPA) avec API REST*
Cible	*Gestionnaires de flotte, chauffeurs, direction*
Environnement	*Postes Windows / navigateurs modernes*
Licence	*Propriétaire (déploiement interne)*

2. Objectifs et enjeux

- Centraliser toutes les données du parc (véhicules, conducteurs, contrôles, maintenance) dans une base unique.
- Réduire les coûts en anticipant les échéances : assurance, vignette, contrôle technique, permis, interventions.
- Accélérer la production des documents réglementaires : fiches de contrôle et rapports PDF professionnels.
- Fiabiliser les décisions grâce à un tableau de bord de KPIs et des statistiques par carburant, marque et site.
- Garantir la sécurité des données (authentification, rôles, sauvegarde) et la traçabilité des actions (audit).

3. Périmètre fonctionnel — les modules

Module	Fonctionnalités clés
Tableau de bord	*KPIs animés, graphiques canvas (donut, barres, anneau), alertes d'échéances, coûts de maintenance*
Véhicules	*CRUD, recherche + filtres, badges de statut, compte à rebours des échéances, photos avec miniatures, fiche PDF*
Conducteurs	*CRUD, matricule, n° de permis, validité du permis, alerte d'expiration*
Contrôles	*Fiche de contrôle interactive (points d'inspection, 3 états), historique par véhicule*
Maintenance	*Interventions, coûts, garages, notes, échéances prévisionnelles (kilométrage + délai)*

Pneumatiques	*Positions des pneus, marques, tailles, états, dates de montage / remplacement*
Relevés kilométriques	*Historique des relevés par véhicule*
Documents	*Upload de fichiers (carte grise, assurances…) associés à un véhicule*
Rapports & PDF	*Rapport complet du parc, rapport mensuel, fiches de contrôle individuelles (bandeau, tableau, signatures, code-barres)*
Import / Export	*Export CSV/XLSX, import depuis gabarit Excel, sauvegarde SQLite*
Administration	*Utilisateurs, rôles, paramètres d'alertes, données de démonstration, journal d'audit*

4. Exigences fonctionnelles détaillées

Réf.	Exigence
F-01	*L'inscription du premier compte crée automatiquement un administrateur ; les suivants sont des utilisateurs standards.*
F-02	*Connexion par nom d'utilisateur ou e-mail avec mot de passe (8 caractères min., au moins un chiffre).*
F-03	*Session JWT valide 7 jours ; changement de mot de passe authentifié.*
F-04	*Les actions administratives (utilisateurs, paramètres, sauvegarde, démo) sont réservées au rôle admin (403 sinon).*

Réf.	Exigence
F-10	*Création / modification / suppression d'un véhicule (immatriculation unique, marque, modèle, année, type, carburant, kilométrage).*
F-11	*Suivi des échéances assurance, vignette et contrôle technique avec jours restants et niveaux d'alerte.*
F-12	*Upload de photo avec génération automatique d'une miniature (vignette 320 px) pour accélérer l'affichage des listes.*
F-13	*Recherche (immatriculation, marque, modèle, affectation) et filtres (statut, carburant, site).*

Réf.	Exigence
F-20	*CRUD conducteurs avec matricule, coordonnées, permis et sa date de validité.*
F-21	*Saisie de contrôles techniques avec type, résultat, notes et points d'inspection.*
F-22	*Saisie de maintenances (type, date, coût, garage, notes) et calcul des échéances prévisionnelles.*
F-23	*Gestion des pneumatiques par véhicule et des relevés kilométriques.*

Réf.	Exigence

F-30	*Rapport complet du parc, rapport mensuel et fiche véhicule générés en PDF professionnel.*
F-31	*Export des données en CSV / XLSX et import depuis un gabarit Excel.*
F-32	*Sauvegarde complète de la base de données (fichier SQLite).*
F-33	*Journal d'audit des actions (connexions, créations, modifications, suppressions).*
F-34	*Génération de données de démonstration réalistes pour l'évaluation.*

5. Exigences non fonctionnelles

Catégorie	Exigence
Sécurité	*Mots de passe hachés PBKDF2-SHA256 (sel aléatoire) ; JWT HS256 avec clé secrète générée hors code ; SQL 100 % paramétré (anti-injection) ; validation et échappement des entrées côté serveur et client.*
Performance	*Temps de réponse API < 300 ms en conditions normales ; miniatures d'images pour des listes légères ; serveur WSGI multi-threadé (waitress).*
Disponibilité	*Application locale auto-hébergée ; sauvegardes de base à la demande ; démarrage en un clic (run.bat).*
Compatibilité	*Python 3.9+ ; navigateurs modernes (Chrome, Edge, Firefox) ; Windows 10/11.*
Maintenabilité	*Code séparé par couches (db, auth, API, PDF, import) ; tests automatisés pytest ; documentation API.*
Ergonomie	*Design sombre premium, glassmorphism, animations, interface entièrement responsive.*

6. Architecture technique et technologie

Frontend	*SPA vanilla JavaScript (h() / rendu par composants), HTML5, CSS3 (design system, glassmorphism), Canvas pour les graphiques*
Backend	*Flask 3 (Python) — API REST*
Base de données	*SQLite (bibliothèque standard, zéro dépendance), schéma versionné avec migrations*
Authentification	*JWT (PyJWT) + Werkzeug (PBKDF2-SHA256)*
PDF	*ReportLab (rapports, fiches, code-barres)*
Images	*Pillow (miniatures 320 px, EXIF)*
Import/Export	*XLSX/CSV en bibliothèque standard (zipfile + XML)*
Serveur	*Waitress (WSGI production, multithreadé)*
Qualité	*pytest (68 tests) + CI GitHub Actions (Python 3.10–3.12)*

7. Modèle de données

Table	Rôle principal
users	*Comptes : identifiant, e-mail, mot de passe haché, rôle (admin / user)*

vehicles	*Véhicules : immatriculation, marque, modèle, type, carburant, kilométrage, échéances, statut, photo + miniature*
drivers	*Conducteurs : nom, prénom, matricule, contact, permis et validité*
controls	*Contrôles techniques : véhicule, conducteur, date, type, résultat, points d'inspection*
maintenance	*Interventions : véhicule, type, date, coût, garage, notes*
tires	*Pneumatiques par véhicule : position, marque, taille, état, dates*
odometer	*Relevés kilométriques : véhicule, date, kilomètres, note*
documents	*Documents attachés aux véhicules*
settings	*Paramètres : seuil d'alerte (jours), intervalle de prévision (km)*
audit_log	*Journal des actions de l'application*

8. API REST

Toutes les routes (hors authentification) exigent l'en-tête Authorization: Bearer <jwt>. Les erreurs sont renvoyées en JSON { "error": "message" }. Voir la documentation complète dans backend/API.md.

Méthode	Route	Description
POST	*/api/auth/register*	*Création de compte (1er compte = admin)*
POST	*/api/auth/login*	*Connexion → { token, user }*
GET	*/api/health*	*Ping de disponibilité (public)*
GET	*/api/stats*	*KPIs, répartitions, coûts, alertes*
GET/POST	*/api/vehicles*	*Liste / création de véhicules*
PUT/DELETE	*/api/vehicles/<id>*	*Modification / suppression*
POST/DELETE	*/api/vehicles/<id>/photo*	*Upload photo + miniature / suppression*
GET/POST	*/api/drivers, /api/controls, /api/maintenance*	*CRUD par module*
GET	*/api/maintenance/forecast*	*Échéances prévisionnelles*
GET	*/api/parc/pdf, /api/reports/monthly/pdf, /api/vehicles/<id>/pdf*	*Rapports PDF*
GET	*/api/export, /api/import/template, POST /api/import*	*Import / export*
GET	*/api/backup*	*Sauvegarde SQLite (admin)*

9. Design, ergonomie et expérience utilisateur

- Thème sombre « aurora » : blobs dégradés flottants + grille, cartes en verre (glassmorphism, backdrop blur).
- Préchargeur animé, compteurs et graphiques animés au canvas (donut, barres, anneau).
- Modales avec effet ressort, toasts de confirmation, badges clignotants sur échéances critiques.

- Tables avec survol, actions contextuelles (icônes), entièrement responsive (sidebar coulissante sur mobile).
- État vide explicite pour chaque module (illustrations + libellés).

10. Déploiement et exploitation

Étape	Action
1. Prérequis	*Python 3.9+ installé (cocher « Add Python to PATH »)*
2. Installation	*Exécuter setup.bat (crée le venv et installe les dépendances)*
3. Lancement	*Exécuter run.bat → ouvre http://127.0.0.1:5000 et démarre le serveur waitress*
4. Premier compte	*La première inscription crée le compte administrateur*
5. Données de démo	*Paramètres → Données de démonstration (facultatif)*
6. Production	*Reverse-proxy HTTPS (Nginx/Caddy), CORS restreint, debug=False*

11. Assurance qualité et tests

Sujet	Détail
Tests automatisés	*68 tests pytest : authentification, véhicules (dont vignette photo), maintenance, contrôles, pneus, conducteurs, statistiques, calendrier, notifications, paramètres, sauvegarde, démo, export, PDF*
Isolation	*Base SQLite temporaire par session de test (aucune donnée réelle touchée)*
Intégration continue	*GitHub Actions : Python 3.10, 3.11 et 3.12 à chaque push / pull request*
Exécution locale	*test.bat ou « python -m pytest » depuis backend/*
Documentation	*backend/API.md décrit toutes les routes, l'authentification et les formats*

12. Planning prévisionnel

Phase	Contenu	Estimation
1. Cadrage	*Expression du besoin, maquettage, choix techniques*	*1 semaine*
2. Socle	*Authentification, base de données, architecture API*	*1 semaine*
3. Modules cœur	*Véhicules, conducteurs, contrôles, maintenance*	*2 semaines*
4. Reporting	*PDF, import/export, sauvegarde, tableau de bord*	*1,5 semaine*
5. Qualité	*Tests, CI, documentation, recette*	*1 semaine*

13. Évolutions futures

- Application mobile (PWA) et mode hors-ligne pour les contrôles terrain.

- Alertes automatiques par e-mail / SMS à l'approche des échéances.
- Géolocalisation et suivi en temps réel des véhicules (flotte connectée).
- Facturation, gestion des carburants (pleins) et rapport de consommation.
- API publique avec clés d'accès et webhooks de synchronisation.
- Multitenant (plusieurs sociétés isolées) et authentification SSO.

14. Risques et mitigations

Risque	Mitigation
Perte de données	*Sauvegardes à la demande ; plan d'automatisation des sauvegardes*
Échéances manquées	*Alertes visuelles multi-niveaux + module calendrier*
Adoption limitée	*Données de démo, interface intuitive, formation courte*
Concurrence d'accès	*SQLite mono-écriture ; bascule possible sur PostgreSQL*
Sécurité	*Rôles, JWT, mots de passe hachés, audit ; revue périodique*

15. Structure du projet

backend/ — app.py (API REST), auth.py (JWT), db.py (SQLite + migrations), pdf.py (ReportLab), imp_exp.py / io_xlsx.py (import-export), wsgi.py (serveur waitress), tests/ (pytest), API.md, requirements.txt*

frontend/ — index.html (SPA), css/style.css (design system), js/app.js (application), ts/ (source TypeScript optionnelle)

Racine — setup.bat, run.bat, test.bat, .github/workflows/tests.yml (CI), data/ et uploads/ (générés), livrables/ (documents)

Document généré le 09/08/2026 — NexFleet — Gestion de parc automobile.